Pflichtenheft

Projekt: ParVer

Version: 0.1.0

Status: In Bearbeitung

Datum: 27.02.2026

Autor: Erik Pförtner

Aktenzeichen: EPSPLG-10007

Zwischen:

AFBB Akademie für berufliche Bildung gGmbH, ansässig in **Güntzstraße 1, 01069 Dresden**, im Folgenden "Auftraggeber" genannt.

Und:

Splatgames.de Software, eine Organisation ohne derzeitige offizielle Rechtsform, vertreten durch den Inhaber **Erik Pförtner**, mit Geschäftssitz in **Hauptstraße 43, 2463 Pottenstein, Österreich**, im Folgenden "Auftragnehmer" genannt.

Inhalt

1. Zielbestimmung ... 4
1.1. Ziel des Projektes ... 4
1.2. Zielgruppen ... 4
2. Produkteinsatz ... 5
2.1 Anwendungsbereich ... 5
2.2 Betriebsbedingungen ... 5
3. Produktfunktionen ... 6
3.1 Administration und Stammdatenpflege ... 6
3.2 Parkplatzstatus und Belegungslogik ... 6
3.2.1 Differenzierung der Statuswerte ... 6
3.2.2 Freigabeprozess durch Parkplatzinhaber ... 6
3.2.3 Belegungsprozess durch Parkplatzsuchende ... 7
3.3 Benachrichtigungssystem (Opt-in) ... 8
3.4 Visuelle Rasterdarstellung ... 8
3.5 Meldewesen bei Abweichungen ... 8
4. Nicht-funktionale Anforderungen ... 9
4.1 Benutzerfreundlichkeit ... 9
4.2 Plattformen ... 9
4.3 Verfügbarkeit und Performance ... 9
5. Erweiterungsoptionen (Optional) ... 10
5.1 Zeitliche Reservierung und Historie ... 10
5.2 Analyse und Statistik ... 10
5.3 Systemintegration und Identifikation ... 10
6. Technische Realisierung ... 11
6.1 Software-Architektur und Tech-Stack ... 11
6.2 Build- und Deployment-Verfahren ... 11
6.3 Fehlerbehandlung und Validierung (Backend) ... 12
7. Benutzeroberfläche ... 13
7.1 Visuelles Design und Farbkonzept ... 13
7.2 Hauptansichten der App ... 13
7.3 Interaktionsdesign und Meldewesen ... 13

7.4 Fehlermeldungen und User-Feedback 14
8. Zusammenfassung 14

1. Zielbestimmung

1.1. Ziel des Projektes

Das primäre Ziel des Projekts „ParVer“ ist die Maximierung der Auslastung organisationsinterner Parkflächen. Durch eine digitale Lösung soll der Leerstand, der durch die feste Zuordnung von Stellplätzen bei gleichzeitiger Abwesenheit der Inhaber entsteht, minimiert werden. Die Software ermöglicht eine dynamische Statusverwaltung, bei der Parkplatzinhaber ihre Plätze aktiv für andere berechtigte Personen freigeben können.

Die Kernziele umfassen:

- Implementierung eines Systems zur Statusänderung (Belegt/Freigegeben).
- Bereitstellung einer Echtzeit-Übersicht der Stellplatzverfügbarkeit.
- Automatisierte Benachrichtigung von Parkplatzsuchenden bei Kapazitätsfreigabe.
- Zentrale Verwaltung von Nutzerrollen und Platzkontingenten.

1.2. Zielgruppen

Die Anwendung ist für den Einsatz innerhalb einer geschlossenen Organisation konzipiert und unterscheidet zwischen drei spezifischen Benutzergruppen:

- **Administratoren:** Diese pflegen die Stammdaten, verwalten die Nutzerkonten und konfigurieren die zwei Parkplatz-Raster zur strukturellen Aufteilung der Stellflächen.
- **User (Benutzer):** Diese Gruppe umfasst alle regulären Anwender der App, wobei systemseitig zwischen zwei funktionalen Ausprägungen unterschieden wird:
    - **Parkplatzinhaber:** User mit einem fest zugeordneten Stellplatz, welche die Berechtigung besitzen, diesen bei Nichtbedarf freizugeben oder wieder als belegt zu markieren.
    - **Parkplatzsuchende:** User ohne eigenen Stellplatz, welche die Raster-Übersicht einsehen, **verfügbare Kapazitäten innerhalb der freigegebenen Zeitfenster** buchen und sich über das „Opt-in“-Verfahren benachrichtigen lassen.

2. Produkteinsatz

2.1 Anwendungsbereich

Die Anwendung wird primär im täglichen Betrieb der Organisation eingesetzt, um die Parkplatzbelegung zu koordinieren. Die Nutzung erfolgt durch die Mitarbeiter (User) direkt vor Ort oder im Vorfeld der Anreise zum Arbeitsplatz, um den Status der Stellflächen zu prüfen oder freizugeben.

2.2 Betriebsbedingungen

Die Software wird als plattformunabhängige **PWA (Progressive Web App)** bereitgestellt. Dies ermöglicht eine native-ähnliche Erfahrung auf mobilen Endgeräten ohne die Notwendigkeit einer App-Store-Distribution.

Technische Anforderungen:

- **Endgeräte:** Smartphone oder Tablet mit aktuellem mobilem Browser (Safari unter iOS, Chrome/Samsung Internet unter Android).
- **Netzkonnektivität:** Für die Echtzeit-Aktualisierung des Parkplatzstatus und den Empfang von Benachrichtigungen ist eine aktive Internetverbindung (Mobilfunk oder WLAN) zwingend erforderlich.
- **Installation:** Die App kann über die "Zum Home-Bildschirm hinzufügen"-Funktion des Browsers als Icon installiert werden und erhält dadurch Zugriff auf Vollbilddarstellung und Web-Push-Funktionalitäten.

3. Produktfunktionen

3.1 Administration und Stammdatenpflege

In diesem Bereich werden die grundlegenden Daten der Anwendung verwaltet. Nur User mit Administrator-Rechten haben Zugriff auf diese Funktionen:

- **Nutzerverwaltung:** Anlegen, Bearbeiten und Löschen von User-Profilen sowie die Zuweisung der Rollen (Inhaber/Suchende).
- **Parkplatzverwaltung:** Definition der vorhandenen Stellplätze und deren eindeutige Zuordnung zu den jeweiligen Parkplatzinhabern.
- **Raster-Management:** Konfiguration der zwei Raster (Zonen/Bereiche), um die Parkplätze logisch zu gruppieren und die visuelle Übersicht zu strukturieren.

3.2 Parkplatzstatus und Belegungslogik

Dieser Abschnitt definiert die Zustände der Stellplätze sowie die Interaktionsmöglichkeiten für Inhaber und Suchende, um eine lückenlose Koordination der Parkflächen zu gewährleisten.

3.2.1 Differenzierung der Statuswerte

Zur visuellen Steuerung werden drei primäre Zustände unterschieden:

- **Rot (Belegt / Reserviert):** Der Parkplatz ist im System vorhanden, aber aktuell nicht freigegeben (Standardstatus) oder bereits durch einen Suchenden für einen spezifischen Zeitraum belegt.
- **Grün (Verfügbar):** Der Parkplatz wurde vom Inhaber aktiv freigegeben und steht zur Belegung durch berechtigte Suchende bereit.
- **Grau (Nicht systemrelevant):** Diese Parkflächen nehmen grundsätzlich nicht am digitalen Freigabeprozess teil (z. B. Besucherparkplätze).

3.2.2 Freigabeprozess durch Parkplatzinhaber

Inhaber können ihre fest zugeordneten Plätze wie folgt verwalten:

- **Zeitsteuerung:** Die Freigabe erfolgt wahlweise tagesbezogen für den aktuellen Tag oder zeitbegrenzt über einen definierten Zeitraum mehrerer Tage.
- **Rücknahmelogik:** Der Inhaber kann eine Freigabe für zukünftige Tage jederzeit widerrufen.
- **Informationspflicht:** Erfolgt die Rücknahme für einen Zeitraum, der bereits von einem Suchenden belegt wurde, löst das System eine automatisierte Benachrichtigung an die betroffene Person aus.

3.2.3 Belegungsprozess durch Parkplatzsuchende

Um die Auslastung zu maximieren, können Suchende aktiv auf freigegebene Kapazitäten zugreifen:

- **Aktive Belegung:** Ein Suchender wählt in der Rasterdarstellung einen „grünen“ Parkplatz aus und beansprucht diesen über die Funktion **„Jetzt belegen“**.
- **Zeitwahl:** Der Suchende legt den benötigten Zeitraum (Von-Bis) innerhalb der vom Inhaber gewährten Freigabe fest.
- **Überbuchungsschutz:** Das System validiert die Anfrage in Echtzeit. Eine Belegung über das Ende des Freigabezeitraums hinaus oder eine Überschneidung mit bereits bestehenden Buchungen wird technisch unterbunden.
- **Statusaktualisierung:** Mit Abschluss des Belegungsvorgangs wechselt der Status für den gewählten Zeitraum systemweit auf „Rot“, um eine Doppelbelegung auszuschließen.

3.3 Benachrichtigungssystem (Opt-in)

Um Suchende aktiv über freie Kapazitäten zu informieren, wird ein Push-Dienst implementiert:

- **Abonnement:** Parkplatzsuchende können sich aktiv als "suchend" markieren (Opt-in).
- **Automatisierte Push-Nachricht:** Sobald ein Inhaber seinen Status auf "Grün" setzt, versendet das System eine Benachrichtigung an alle angemeldeten Suchenden.
- **Optionaler E-Mail-Versand:** Ergänzend zur Push-Benachrichtigung kann eine Benachrichtigung per E-Mail erfolgen.

3.4 Visuelle Rasterdarstellung

Zur schnellen Orientierung bietet die App eine Übersicht der zwei definierten Raster:

- **Zonen-Übersicht:** Darstellung der zwei Bereiche (die Zonen).
- **Kapazitätsanzeige:** Jedes Raster zeigt die Anzahl der insgesamt vorhandenen Plätze sowie das aktuelle Verhältnis von verfügbaren (grünen) zu belegten (roten) Plätzen an.
- **Detailansicht:** Suchende können in ein Raster hineinzoomen, um die genauen Positionen der freien Plätze zu sehen.

3.5 Meldewesen bei Abweichungen

- **Fehlbelegung melden:** Stellt ein Suchender vor Ort fest, dass ein als „Frei“ oder „Reserviert“ markierter Platz blockiert ist, kann er dies über einen dedizierten Dialog in der App melden.

4. Nicht-funktionale Anforderungen

4.1 Benutzerfreundlichkeit

Die Anwendung muss so gestaltet sein, dass sie ohne vorherige Schulung intuitiv bedienbar ist.

- **Farbcodierung:** Zur schnellen visuellen Orientierung wird eine strikte Farbcodierung nach dem Ampelsystem (Rot für belegt, Grün für verfügbar) verwendet.
- **Struktur:** Die Aufteilung in zwei Raster ermöglicht eine übersichtliche Darstellung auch bei einer größeren Anzahl an Parkplätzen.
- **Bedienung:** Das Interface ist auf minimale Klickpfade optimiert, um Freigaben oder Reservierungen in Sekunden durchführen zu können.

4.2 Plattformen

Die Software wird plattformunabhängig entwickelt, um eine maximale Reichweite innerhalb der Organisation zu erzielen.

- **PWA-Technologie:** Die App wird als Progressive Web App (PWA) realisiert, wodurch sie sowohl auf iOS- als auch auf Android-Endgeräten direkt über den Browser ohne Store-Zwang genutzt werden kann.
- **Responsivität:** Das Design passt sich automatisch an verschiedene Bildschirmgrößen mobiler Endgeräte an.

4.3 Verfügbarkeit und Performance

Für die Akzeptanz des Systems ist die Aktualität der Daten kritisch.

- **Echtzeit-Aktualisierung:** Der Status der Parkplätze muss sich bei Änderungen durch andere User ohne manuelles Neuladen der Seite aktualisieren.
- **Benachrichtigungsgeschwindigkeit:** Push-Meldungen über freigegebene Plätze müssen unmittelbar nach der Statusänderung bei den Suchenden eingehen.
- **Stabilität:** Das System muss so konzipiert sein, dass es auch bei gleichzeitigem Zugriff vieler User (z. B. zu Schichtbeginn) performant reagiert.

5. Erweiterungsoptionen (Optional)

In diesem Kapitel werden Funktionen beschrieben, die über den Kernumfang hinausgehen und die Effizienz des Systems langfristig steigern können.

5.1 Zeitliche Reservierung und Historie

- **Reservierungssystem:** Ermöglichung einer zeitlichen Reservierung von Parkplätzen durch Suchende, um Planungssicherheit zu schaffen.
- **Nutzungshistorie:** Protokollierung der Parkplatznutzung, um den Anwendern eine Übersicht ihrer vergangenen Freigaben oder Beanspruchungen zu bieten.

5.2 Analyse und Statistik

- **Auslastungsstatistiken:** Grafische Aufbereitung der Parkplatzbelegung für Administratoren, um Engpässe oder dauerhaften Leerstand zu identifizieren.
- **Optimierungspotenziale:** Ableitung von Trends aus den Nutzungsdaten zur besseren Planung der Parkraumkapazitäten.

5.3 Systemintegration und Identifikation

- **Kalenderintegration:** Automatische Freigabe des Stellplatzes basierend auf Kalendereinträgen (z. B. bei Abwesenheit durch Urlaub oder Krankheit).
- **Identifikationsverfahren:** Integration von QR-Codes oder einer Kennzeichenbindung zur Verifizierung der Berechtigung direkt am Stellplatz.

6. Technische Realisierung

6.1 Software-Architektur und Tech-Stack

Die Anwendung wird als Client-Server-System umgesetzt, wobei eine moderne Trennung zwischen Frontend und Backend erfolgt, die zur Laufzeit jedoch als eine Einheit agiert:

- **Frontend:** Entwicklung einer Progressive Web App (PWA) auf Basis von React. Dies garantiert die geforderte Plattformunabhängigkeit für iOS und Android sowie eine native-ähnliche Benutzererfahrung.
- **Backend:** Realisierung mit Java und dem Spring Boot Framework. Die Geschäftslogik, die Rollenprüfung und die Statusverwaltung werden hier zentral verarbeitet.
- **Datenhaltung:** Einsatz einer relationalen Datenbank zur persistenten Speicherung der Parkplatzstatus (Rot, Grün, Grau) und der Nutzerzuordnungen.

6.2 Build- und Deployment-Verfahren

Der Fokus liegt auf einem hocheffizienten und portablen Auslieferungsprozess:

- **Integrierter Build-Prozess:** Das React-Frontend wird kompiliert und als statische Ressource (Static Resource) direkt in das Java-Archiv (JAR) des Backends eingebunden. Dies ermöglicht den Betrieb der gesamten Applikation über eine einzige ausführbare Datei.
- **Containerisierung:** Zur Vereinfachung der Bereitstellung wird die Anwendung in ein Docker-Image verpackt. Dies ermöglicht schnelle Deployments und eine konsistente Ausführung auf unterschiedlichen Serverumgebungen.
- **Echtzeit-Synchronisation:** Die geforderte Echtzeit-Aktualisierung der Parkplatzstatus wird im Backend über WebSockets oder ähnliche Protokolle realisiert, um Änderungen ohne manuellen Refresh im Frontend anzuzeigen.

6.3 Fehlerbehandlung und Validierung (Backend)

Um die Datenintegrität und Systemstabilität zu gewährleisten, werden folgende Mechanismen implementiert:

- **Eingabevalidierung:** Alle vom Frontend eingehenden Daten (z. B. Freigabezeiträume) werden im Spring-Boot-Backend gegen das definierte Datenmodell geprüft.
- **Status-Logik-Prüfung:** Das System verhindert logische Fehlzustände, wie beispielsweise die Freigabe eines bereits freigegebenen Platzes oder die Reservierung eines nicht existierenden Stellplatzes.
- **Exception Handling:** Unvorhergesehene Serverfehler werden durch einen zentralen GlobalExceptionHandler abgefangen, um dem Client eine definierte Fehlermeldung im JSON-Format zurückzugeben, statt den Dienst zu unterbrechen.
- **Datenbank-Konsistenz:** Transaktionale Vorgänge stellen sicher, dass bei einem Verbindungsabbruch während eines Statuswechsels keine inkonsistenten Daten (z. B. ein Platz ohne eindeutigen Status) entstehen.

7. Benutzeroberfläche

7.1 Visuelles Design und Farbkonzept

Das Interface nutzt die im Anforderungsprofil vorgegebene Farbcodierung zur sofortigen Erfassung des Parkplatzstatus:

- **Rot:** Der Parkplatz ist grundsätzlich vorhanden, aber aktuell nicht freigegeben (Standardstatus für alle freigebenden Plätze).
- **Grün:** Der Parkplatz wurde vom Parkplatzinhaber aktiv freigegeben und ist somit für andere Personen verfügbar.
- **Grau (Nicht hervorgehoben):** Kennzeichnung für Parkplätze, die grundsätzlich nicht zu den freigebenden Plätzen im System gehören.
- **Raster-Logik:** Die Visualisierung erfolgt in zwei Raster (die beiden Zonen der Stellplätze), um eine schnelle visuelle Orientierung zu ermöglichen, wo grundsätzlich Parkplätze verfügbar sein könnten.

7.2 Hauptansichten der App

Die auf React basierende PWA stellt die notwendigen Ansichten bereit, um die Anforderungen umzusetzen:

- **Raster-Dashboard:** Zentrale Übersicht der zwei Zonen. Jedes Raster zeigt die Anzahl der Parkplätze sowie eine Statusübersicht in Rot/Grün an.
- **Inhaber-Ansicht:** Ermöglicht dem Parkplatzinhaber, den Status des eigenen Platzes einzusehen und diesen tagesbezogen oder zeitlich begrenzt freizugeben.
- **Such-Ansicht:** Zeigt die verfügbaren Plätze in der Raster-Übersicht. Bei Klick auf einen grünen Platz öffnet sich ein **Buchungs-Overlay**, in dem der Nutzer seinen gewünschten Zeitraum innerhalb der erlaubten Grenzen festlegt.
- **Administrator-Bereich:** Interface zur Verwaltung der Parkplatzinhaber, Suchenden, der Parkplatz-Zuordnungen sowie der zwei Raster.
- **Bestätigungs-Dialog:** Nach erfolgreicher Belegung erhält der Nutzer ein visuelles Feedback (Toast/Meldung) mit der Bestätigung seines Zeitraums.

7.3 Interaktionsdesign und Meldewesen

- **Benachrichtigungs-Opt-in:** Suchende können sich über ein Opt-in für Push-Nachrichten (oder optional E-Mail) registrieren, die ausgelöst werden, sobald mindestens ein Parkplatz freigegeben wurde.
- **Lösung für Fehlbelegungen:** Die UI bietet eine Möglichkeit an, falls ein Parkplatz unrechtmäßig belegt ist.
- **Echtzeit-Feedback:** Statusänderungen durch Inhaber führen zu einer sofortigen Aktualisierung der Rasteransicht für alle Nutzer.

7.4 Fehlermeldungen und User-Feedback

Die PWA kommuniziert Fehlerzustände klar und verständlich an den Endnutzer:

- **Visuelles Feedback:** Bei fehlgeschlagenen Aktionen (z. B. Netzwerkfehler beim Freigeben) erscheint eine Benachrichtigung (Toast oder Alert) in der App.
- **Validierungshinweise:** Fehleingaben in Formularen (z. B. beim Admin-Interface) werden direkt am betroffenen Feld markiert.
- **Offline-Indikator:** Da die PWA eine aktive Internetverbindung benötigt, wird der User gewarnt, wenn die Verbindung unterbrochen ist und Aktionen derzeit nicht synchronisiert werden können.
- **Meldung unrechtmäßiger Belegung:** Wie im Anforderungsprofil gefordert, bietet die UI einen spezifischen Dialog an, um Abweichungen zwischen Systemstatus und realer Parkplatzbelegung zu melden.

8. Zusammenfassung

Die Parkapp „ParVer“ schafft Transparenz über verfügbare Parkplätze, reduziert Leerstand und verbessert die Nutzung bestehender Ressourcen. Durch klare Rollen, einfache Freigabeprozesse und eine visuelle Rasterübersicht wird der organisatorische Aufwand minimiert. Die technische Umsetzung als containerisierte Spring-Boot-Anwendung mit integriertem React-Frontend gewährleistet eine performante Bereitstellung als PWA.

Pflichtenheft

ParVer – 0.1.0

Datum (Tag der Unterschrift)	[Name des Auftraggebers] Auftraggeber (Unterschrift)
Datum (Tag der Unterschrift)	Erik Pförtner Auftragnehmer (Unterschrift)